

Model Checking Concurrent Programs with SPIN

RW796 Software Verification and Analysis

Gabriella Scott
25935453

September 1, 2026

Introduction

This report covers Promela models of two concurrent programs and the verification of their correctness properties using Spin. The first program is a first-come first-served (FCFS) process scheduler, one of three policies supported by a shared-memory process manager. A team of OpenMP threads dispatches processes from a common ready queue, moves them to a waiting queue when a resource is unavailable, and updates a shared resource table, with nested locks protecting these structures and new processes arriving while the threads are already running. The second program is a distributed Othello engine running a parallel alpha-beta search. An MPI master hands candidate moves out to worker processes as they report back, propagates a shared alpha bound between them, and reduces the search depth partway through a round when the turn clock runs down. It communicates over three channels: MPI collectives to start a round, point-to-point messages to distribute and collect work, and a socket to the referee.

Parts of each program were omitted, some behaviour was replaced with nondeterministic choices, and the remaining parameters were fixed to small values. These three moves do not affect the results in the same way, so it helps to separate them with the usual pair of terms. An abstraction is *sound* for a property if every counterexample it reports corresponds to a real execution of the implementation, meaning there are no false alarms. It is *complete* if every violation the implementation can commit also appears in the model, meaning there are no missed defects.

Replacing concrete behaviour with a nondeterministic choice adds executions rather than removing them, which is why Promela treats nondeterminism as the standard way to build an abstraction [3]. That direction keeps completeness but puts soundness at risk, because the model can now do things the implementation never would. Fixing the parameters to small values and leaving components out works the other way: it removes executions, so a counterexample that survives is still a real one, but a property that passes has only been shown to pass inside the bound. Neither model here is purely one or the other, so both directions are handled explicitly. Every counterexample is read back against the C source before it is called a defect, and every passing result is reported with its configuration attached rather than as a general claim. Each abstraction is justified where it is introduced, and Section 1.1 and Section 2.1 give the specific limits for each program.

The analysis found defects in both programs. For the scheduler, 6 of the 7 checks failed, with the counterexamples tracing back to two faults in `manager.c`: a resource request that does not check whether the requester already owns the resource, and a dispatch loop that accesses shared process state without holding a lock. For the Othello engine, 4 of the 5 checks failed, with 3 faults identified in `my_player.c`, namely an initialiser escaping from the minimax minimising branch as if it were a score, comparisons between scores calculated at different search depths, and two referee message types for which the master loop has no corresponding branch. The last

fault did not occur in any recorded match, illustrating the value of model checking in finding behaviours that testing may not expose.

Implementation under analysis (Program 1): [\[TODO: repo URL\]](#)

Implementation under analysis (Program 2): [\[TODO: repo URL\]](#)

Promela models, properties and verification records: https://github.com/Gabriella-Scott/PA_Assignments

1 Program 1: FCFS Process Scheduler

1.1 The Promela Model

The model abstracts the FCFS path of `manager.c`: `schedule_fcfs` together with `execute_instr`, `request_resource`, `release_resource`, `enqueue_pcb`, `dequeue_pcb` and `terminate`. It consists of an `init` process that seeds the ready queue, three inlines standing for `terminate`, `request_resource` and `release_resource`, and a `Worker` proctype instantiated once per scheduling thread. The other two policies are not modelled: all three sit on the same queues, locks and resource table, so including them would have multiplied the state space without reaching a different class of defect.

Four constants fix the size of the model, each the smallest value that still shows the behaviour of interest. `NUM_WORKERS` is 2, because a single thread cannot interleave with anything. `NUM_PROCS` is 3, because with 2 processes one running and one blocked leaves the ready queue empty, and a third is needed before a process can be dispatched while another is already blocked. `NUM_RESOURCES` is 2, the smallest number admitting a circular wait [8]. `MAX_INSTR` is 3, enough for a process to request a resource, release it and request again, so that a release can wake a blocked process. Identifiers start at 1 so that 0 can stand for NULL and `resource_owner[r] == 0` reads exactly as `resource->allocated == NULL` does. Only `MAX_INSTR` is wrapped in an `#ifndef` guard, since it is the one parameter varied between runs; supplying it with `spin -D` means every result in Section 1.3 comes from the same committed file. Table 1 sets out the correspondence between the implementation and the model.

OpenMP nested locks were used to protect the shared structure in the implementation, which let a thread reacquire a lock it already holds [7]; `schedule_fcfs` takes `i_lock` and then calls `execute_instr`, which takes it again. The model represents no locks at all, only the `atomic` blocks standing for the regions they protect. That is sound provided the locks cannot deadlock against each other, which is checkable by inspection: every FCFS path acquires them in the same order, and `release_resource`, the only function holding two queue locks at once, always takes `q_locks[1]` before `q_locks[0]`. With a consistent acquisition order no circular wait between locks is possible [8]. One caveat to note is that a blocking statement inside an `atomic` sequence causes Spin to drop atomicity rather than report an error [1], so the sends on `ready_q` and `waiting_q` could in principle interleave, but both channels are sized at `NUM_PROCS` and can only fill if a PCB is enqueued twice, which is what `no_dup_ready` exists to rule out.

What the model leaves outside an `atomic` block matters more. The assignment `running_p->state = RUNNING`, the loop guard `while (running_p->state == RUNNING)` and the test `if (running_p->state == WAITING)` all touch shared PCB state with no lock held, since `i_lock` is released as soon as `execute_instr` returns. Those statements sit outside any atomic block in the `Worker` loop, so `pan` is free to interleave the second worker there. Wrapping them in `atomic` for tidiness would have left the model unable to find the defect in Section 1.3.

Listing 1: Unprotected reads in `schedule_fcfs`, abridged

```
running_p->state = RUNNING; /* no lock held */
while (running_p->state == RUNNING) /* no lock held */
{
    ...
}
```

Table 1: Mapping between the implementation and the model.

<code>manager.c</code>	Model	Reason
Thread in <code>schedule_fcfs</code>	<code>active</code> <code>[NUM_WORKERS]</code> proctype <code>Worker</code>	The parallel region runs identical threads over the same loop, so one proctype replicated twice covers every thread the scheduler creates.
<code>readyq, waitingq</code>	Buffered channels <code>ready_q, waiting_q</code>	FIFO order is the only behaviour of the linked lists FCFS depends on. <code>waiting_q</code> also carries the resource id, which the implementation reads from the instruction record.
<code>terminatedq</code>	<code>pcb_state[] == TERMINATED</code>	Nothing is ever dequeued from it, so membership carries no information the state field does not already hold.
<code>pcb_t.state</code>	<code>pcb_state[]</code>	Direct, indexed by process id instead of reached through a pointer.
<code>next_instruction</code>	<code>instr_count[]</code>	The instruction list becomes a bounded counter, with <code>next_instruction == NULL</code> represented by <code>instr_count == MAX_INSTR</code> .
<code>resource_t.allocated</code>	<code>resource_owner[]</code>	The list searched by <code>strcmp</code> becomes an array indexed by id, and <code>allocated == NULL</code> becomes owner 0.
<code>q_locks[0], omp critical, i_lock, r_lock</code>	<code>atomic blocks</code>	Mutual exclusion is all the locks provide here, and <code>atomic</code> supplies it without modelling acquisition [1]. Only the regions the implementation actually protects are made atomic.
<code>terminate()</code>	<code>check_terminate</code> inline	The same test, evaluated over the state array rather than by walking the queues.

```

omp_set_nest_lock(&i_lock);
...
execute_instr(running_p);
omp_unset_nest_lock(&i_lock); /* lock released here */

load_new_processes();

if (running_p->state == WAITING) /* no lock held */
{
    break;
}
running_p->next_instruction = running_p->next_instruction->next;
...
}

```

Two arrays in the model do not have direct counterparts in `manager.c`; `executing[]` keeps track of how many workers are currently processing a PCB, while `in_ready[]` tracks how many times a PCB appears in the ready queue. Since neither is used in a guard, they cannot enable or block any transitions; only provide the state needed by `pcb_mutex` and `no_dup_ready`.

The instruction stream is where the model makes the biggest abstraction, because the implementation reads instructions from a linked list created by the loader, where each instruction has a type and resource name. The model does not include these instruction records; instead, `select` non-deterministically chooses a resource, while a two-branch `if` non-deterministically chooses between a request and a release [3]. Because `pan` explores both branches, the model covers every possible instruction sequence of up to `MAX_INSTR` steps, including sequences that the actual loader would not produce. Table 2 lists what was left out and what each omission costs.

Table 2: What the model leaves out, and what each omission costs.

Omitted	Cost
Process names, <code>strcmp</code> lookup	None → Resources are found by index (instead of by walking a list).
Logging	None → It writes to a file and touches no scheduling state.
<code>omp barrier, single</code>	None → They run after every process has terminated, so no property here can distinguish them.
The "resource does not exist" path	Small but real → The implementation reaches the same <code>enqueue_pcb(..., WAITING)</code> branch whether a resource is held by another process or absent entirely; the model represents only the first.
Dynamic arrivals and <code>a_lock</code>	Not free → <code>load_new_processes</code> can add to the ready queue while <code>terminate()</code> is counting it, so omitting arrivals removes a race the model cannot then find.
<code>detect_deadlock</code>	Not free → Deadlock is checked by Spin's invalid end state detection instead, which asks whether the system halted where it should not have rather than whether a circular wait exists.

Three changes were needed because of how Promela works. An `if` statement with no executable guard blocks, so the test after an instruction uses two explicit guards for `WAITING` instead of a guard and an `else`. Spin also doesn't allow `else` to be used with channel operations, so the dequeue uses `nempty(ready_q)` and `empty(ready_q)` as two explicit cases.

A worker can try to dequeue a process before the ready queue has been filled, because the `active` proctypes start at the same time as `init`. The `empty(ready_q)` branch handles this case, but it also causes the simple liveness failures discussed in Section 1.3.

Limits on interpretation

The model was written by hand, so a counterexample is treated as a possible defect until its interleaving has been checked against the source code. This is done for each finding in Section 1.3. On the other hand, a property that holds only applies to the model configuration of 3 processes, 2 workers, 2 resources, and `MAX_INSTR` steps. One important assumption is that the model treats unprotected reads of `running_p->state` as ordinary reads that see the most recent value written. OpenMP does not guarantee this behaviour without a `flush` [7]. Consequently, the real program may have behaviours that are not represented in the model. This means that the identified defects are a lower bound on the problems with the unlocked dispatch loop.

1.2 Correctness Properties

Seven checks were run: 6 properties written as named `ltl` blocks in `model.pml`, plus a deadlock check that is not an LTL property. Table 3 gives each formula as it appears in the file, where `[]` is $\Box$ and `<>` is $\Diamond$ [4]. 4 are safety properties, which say a bad state is never reached and are violated by a finite prefix, and 2 are liveness properties, which say something good eventually happens and can only be violated by an infinite execution [4]. Promela has no quantifier, so every conjunction over the three process ids is written out by hand.

Table 3: The six LTL properties as written in `model.pml`.

Property	Class	Formula
<code>pcb_mutex</code>	safety	<code>[] (executing[p] <= 1) for each p</code>
<code>no_self_wait</code>	safety	<code>[] (waiting_for[p] == 0 resource_owner[waiting_for[p]] != p)</code>
<code>waiting_consistent</code>	safety	<code>[] (pcb_state[p] != WAITING waiting_for[p] != 0)</code>
<code>no_dup_ready</code>	safety	<code>[] (in_ready[p] <= 1)</code>
<code>all_terminate</code>	liveness	<code><> (pcb_state[1] == TERMINATED && ... && pcb_state[3] == TERMINATED)</code>
<code>ready_dispatched</code>	liveness	<code>[] ((pcb_state[1] == READY) -> <> (pcb_state[1] == RUNNING))</code>

`pcb_mutex` is an important invariant because the PCB represents a unit of work, so only one thread should process it at a time. The dequeue operation is protected, but the code after it is not. This means two threads could enter the instruction loop for the same PCB, causing both to update `next_instruction` and `state`.

`no_self_wait` checks the behaviour of `request_resource`. The function only checks whether `resource->allocated == NULL` and does not check whether the resource is already owned by the requesting process. This results in a process being able to request a resource it already holds and be placed on `waitingq`. Since the process is the one holding the resource, it cannot release it while waiting. Unlike the other safety properties, this failure does not depend on a particular thread interleaving (so it is considered separately).

`waiting_consistent` checks that a PCB in state `WAITING` is actually waiting for a resource, since the state and resource information are updated separately without a lock protecting both

operations. This means a PCB can enter the `WAITING` state without having a resource recorded. It can then remain on the waiting queue without anything being able to wake it. This property is considered together with `pcb_mutex`, because it shows how overlapping access can lead to incorrect state that affects later decisions.

`no_dup_ready` checks the three places that add processes to the ready queue: the long-term scheduler, `load_new_processes`, and `release_resource`. A duplicate entry could cause the same process to be dispatched twice and lead to another `pcb_mutex` violation that cannot be prevented just by locking the dispatch loop.

`all_terminate` is checked separately from the program’s own stopping condition because they don’t ask the same question. `terminate()` returns `TRUE` when all processes have been placed in either the waiting or terminated queues. This means a process that is blocked forever can still allow the workers to exit and the scheduler to report that it has finished.

`ready_dispatched` checks whether the system actually serves processes before stopping. It is only checked for process 1 because the 3 processes are symmetric. FCFS does not use a priority field, and all 3 instruction sequences are generated using the same nondeterministic choice. Therefore, checking one process represents the others and keeps the never claim smaller. This is an assumption based on symmetry rather than a formal proof, and it is the one property in Program 1 that was weakened to reduce the verification cost. Both liveness properties are checked under weak fairness, as discussed in Section 1.3.

Deadlock is checked with Spin’s invalid end state detection, which asks whether any process is stopped somewhere other than a valid halting point [4]. That is broader than asking whether a circular wait exists [8]; it catches a process blocked on a guard that can never become executable, whatever the reason. Since `pan` disables the check while a never claim is active, this run uses a separate claim-free verifier [2].

Three things were deliberately not checked, namely an assertion bounding `instr_count` was removed, because the deadlock trail already shows the same overrun and an active assertion would let other runs stop on it rather than on their named property. `detect_deadlock` is not verified, because it runs inside `#pragma omp single` and contains no concurrency. Lock reentrancy is not modelled, because no property depends on nesting depth, only on which accesses are protected.

1.3 Verification Results and Analysis

Each run selects a specific claim using `./pan -a -N name`. If `-N` is not provided, `pan` uses the first claim in the file by default. The deadlock check uses a second binary compiled from the same generated `pan.c` with `-DNOCLAIM`. This removes the claims from the verifier, rather than from the model itself [2]. The value of `MAX_INSTR` is also supplied to `spin` on the command line, ensuring that all results in Table 4 use the same source file.

Two flags are used throughout the verification. `-m` increases the maximum search depth from the default of 10,000, while `-w26` increases the hash table size to 2^{26} entries. The larger hash table was needed because the default 2^{19} entries caused many hash conflicts at this scale [2]. All runs were performed from the command line using Spin 6.5.2 on a machine with 7,GB of RAM. iSpin was not used.

Six of the seven checks failed, but they do not describe six problems. They collapse into two defects.

The double dispatch race

`pcb_mutex` fails at depth 901 when `executing[]` reaches 2. After checking the counterexample against `manager.c`, the interleaving is as follows. Worker A dequeues process p and runs an instruction requesting a resource held by another process. `request_resource` then sets p to

Table 4: Verification results. Depths and state counts as reported by `pan`.

Property	Config	Result	Depth	States
<code>pcb_mutex</code>	<code>MAX_INSTR 3</code>	violated	901	65 957
<code>no_self_wait</code>	<code>MAX_INSTR 3</code>	violated	379	188
<code>waiting_consistent</code>	<code>MAX_INSTR 3</code>	violated	962	69 699
<code>no_dup_ready</code>	<code>MAX_INSTR 2</code>	holds	n/a	20 861 451
<code>all_terminate</code>	<code>MAX_INSTR 3, no -f</code>	violated	14	25
<code>all_terminate</code>	<code>MAX_INSTR 3, -f</code>	violated	1069	493
<code>ready_dispatched</code>	<code>MAX_INSTR 3, no -f</code>	violated	14	25
<code>ready_dispatched</code>	<code>MAX_INSTR 3, -f</code>	violated	1088	11 372 991
<code>deadlock</code>	<code>MAX_INSTR 3, -DNOCLAIM</code>	violated	504	67 872

`WAITING` and adds it to `waitingq`. A releases `i_lock` and is about to check `running_p->state == WAITING` to decide whether to leave the loop. Before this happens, worker B releases the resource, causing `release_resource` to move `p` back to `readyq` and set its state to `READY`. B then dequeues `p` and starts running it. A reads the updated `READY` state, does not leave the loop, and continues running `p` as well.

`waiting_consistent` fails 61 steps later in the same part of the execution. The trail shows a PCB in the `WAITING` state even though the resource it was waiting for has already been released by the other worker. Since `release_resource` only wakes a waiting process when the resource matches the one it is waiting for, this PCB remains in `waitingq` with no way to return to the ready queue. It also continues holding a resource that it will never release.

The deadlock run is the same defect from the consequence side. Its final state has `instr_count[2]` at 4 with `MAX_INSTR` at 3, leaving a worker stuck at the selection on `instr_count` because neither guard is executable. In `manager.c` that is two threads both running `running_p->next_instruction = running_p->next_instruction->next` for the same PCB, so the second advances a pointer already at the end of the list and dereferences past it. C defines no behaviour there, and a Promela control point with no executable transition is a state with no defined successor, so the abstraction has nothing defined exactly where the implementation has nothing defined.

The fix is to hold a lock across the state test and the branch that follows it, not only across `execute_instr`. Doing that with `i_lock` costs parallelism, since `i_lock` already serialises every instruction in the system; a per-PCB lock is the cheaper version, because then only threads touching the same PCB serialise.

Self deadlock in `request_resource`

This counterexample is much smaller than the others, failing at depth 379 after 188 states. The trail only contains one worker, and `executing[]` never becomes greater than 1, so there is no thread interleaving involved. This is simply a logic error that a single thread can reach on its own. The fix is to add a check before the availability test so that `resource->allocated == cur_pcb` is treated as a successful request. The same `!resource_found` branch is also used when the requested resource does not exist. These are two different cases, so the fix should distinguish between a resource already owned by the process, a resource held by another process, and a resource that does not exist.

This defect also explains both liveness failures and has another consequence that is not checked by the model. `detect_deadlock` searches for cycles in the waiting queue, and a process waiting for a resource that it already owns creates a cycle of length 1. The program would therefore report a deadlock even though it is not the type of deadlock the diagnostic is intended to detect. A single missing check therefore causes a safety violation, 2 liveness violations, and an incorrect deadlock diagnosis.

Liveness and fairness

Both liveness properties also fail without `-f`, at depth 14 after 25 states, and neither of those results is evidence of anything. The cycle they report is one in which `init` evaluates its loop guard and is never scheduled again, so `ready_q` is never seeded and a worker spins on an empty queue forever. No OpenMP runtime passes over a thread that can make progress indefinitely [7], and weak fairness excludes exactly those cycles [4], so every liveness result reported here uses `-f`. Fairness is not free: `ready_dispatched` finishes in 25 states without it and explores 11372991 with it.

The fair `all_terminate` counterexample ends with all three processes `WAITING`, process 1 holding resource 1 while waiting for resource 1, and both workers already exited because `check_terminate` counts every process as blocked. `ready_dispatched` fails for the same reason but exposes a separate design consequence; which is the stopping condition retires the workers as soon as everything is blocked, so a process can still be on the ready path when the system decides it is finished.

The property that holds

`no_dup_ready` was checked exhaustively at `MAX_INSTR 2` with 0 errors over 20861451 states in about 21 seconds, using 2421 MB. The run also reported 0 of 12 states unreachable in `init` and 0 of 119 unreachable in `Worker`, which is what makes the pass worth quoting: every control point was visited, so the result is not an artefact of dead code in the abstraction [5].

The reduced bound is necessary because of the size of the state space. Failed properties are relatively cheap to check because the search stops when the first counterexample is found. However, a property that passes must explore all reachable states. The state space grows by about a factor of 50 for each additional instruction. 420,705 states at `MAX_INSTR 1` and 20861451 at 2, which would result in roughly 10^9 states at 3. With `pan` reporting 96 bytes per state, this would require around 100 GB of memory, compared to the 7 GB available.

The bound of 2 is reasonable because the shortest possible duplicate requires a PCB to be added to the queue twice before it is removed once. This requires one blocking request followed by a wakeup, so two instructions are enough to produce the smallest possible counterexample. Increasing the bound would therefore mainly allow longer versions of the same behaviour. This argument is based on the shortest witness rather than a formal proof.

It is also important to note that the model uses a channel for the ready queue. A channel cannot contain two references to the same PCB in the way that an incorrectly maintained linked list could; this result checks the enqueue and dequeue behaviour rather than the pointer manipulation performed by `enqueue_pcb`.

2 Program 2: MPI Master/Worker Othello Engine

2.1 The Promela Model

The model abstracts the MPI master and worker move evaluation in `my_player.c`: `run_master`, `execute_master`, `send_init_moves`, `send_next_move`, `pop_move` and `run_worker`. The referee is modelled as an environment process, `Referee`, because a Promela model must be closed and the communication flow through `comms.c` is fixed by the framework rather than by us. The board, the search recursion, `evaluate` and the Othello rules are all abstracted away. What survives of `minimax_pruning` is only which of three outcomes a search produces, which is the least the properties in Section 2.2 need.

Three constants fix the size of the model, all wrapped in `#ifndef` guards so they are supplied on the command line. `NUM_WORKERS` is 3, not an arbitrary choice: `common/configs.py` in the Othello project sets `"threads": 4` for this game, so `comm_sz` is 4, one master and three

workers, and the model matches the deployed system. `MAX_MOVES` must exceed `NUM_WORKERS`, or `send_init_moves` empties the move stack before any result comes back and the dynamic work-redistribution branch is never taken. This was not decided in advance; it was caught by `pan`'s unreachable statement report at `NUM_WORKERS 3`, `MAX_MOVES 3`, which named the redistribution send as dead code, the same kind of finding `pan` gave for the scheduler's duplicate-ready check. `MAX_MOVES 4` is the smallest value that fixes it. `MAX_ROUNDS` bounds the `Referee` process so every run terminates; it is 2 by default and varied per run in Section 2.3. Table 5 sets out the correspondence between the implementation and the model.

Table 5: Mapping between the implementation and the model.

my_player.c	Model	Reason
<code>MPI_Bcast(&my_colour)</code> <code>MPI_Bcast(board)</code>	<code>bcast_colour[]</code> , <code>bcast_board[]</code>	One channel per worker for each collective. The board carries no information any property needs, so it is reduced to the round number.
<code>MPI_Send/MPI_Recv</code> with <code>MOVE_TAG</code> , <code>NO_WORK_TAG</code>	<code>to_worker[]</code> , one buffered channel per worker	Direct: the tag distinguishes work from termination exactly as in the implementation.
<code>MPI_Iprobe(MPI_ANY_SOURCE)</code> + <code>MPI_Recv</code>	<code>worker</code> , one shared channel	<code>MPI_ANY_SOURCE</code> does not distinguish senders, so a single channel every worker writes into is a direct abstraction, not a simplification.
<code>move_stack</code> (linked list)	<code>moves_left</code> counter	Nothing else reads the stack and no other process touches it, so only its size can affect any interleaving.
<code>minimax_pruning</code>	<code>evaluate_move</code> in-line	The board, recursion and <code>evaluate</code> are removed. What survives is which of three outcomes a call produces, discussed below.
<code>run_master</code> 's if-chain over <code>comms.h</code> message types	<code>Master's if</code> over <code>mtype</code>	Direct: the same branches are handled, and the same two are not.
The Ingenious Framework referee (reached over a socket via <code>comms.c</code>)	<code>Referee</code> , an environment process	Promela models must be closed [3]; <code>budget</code> bounds the messages so every run terminates.

The headline abstraction is `evaluate_move`, which stands for the call a worker makes into `minimax_pruning` and, through it, the minimising branch of `minimax`. That branch initialises an accumulator to `INT_MAX` and loops over the legal moves at that node:

Listing 2: The minimising branch of `minimax`

```

minEval = INT_MAX;
legal_moves(moves, &number_of_moves, my_colour);
for (int i = 0; i < number_of_moves; i++)
{
    ...
    minEval = minimum(minEval, eval);
    beta = minimum(beta, eval);
    if (beta <= *alpha)
    {

```

```

    break;
  }
}
return minEval;

```

Three outcomes fall out of this: the loop never runs because there are no legal moves at that node, so the function returns its initial value unchanged; the loop runs once and stops, because the pruning test sits at the bottom rather than the top, so a poisoned or reduced-depth call returns its first child's value rather than a true minimum; or the loop runs to completion and returns a genuine evaluation. The model reproduces exactly these three outcomes as three branches of one `if`, resolved non-deterministically wherever more than one applies:

Listing 3: `evaluate_move`, the abstraction of `minimax_pruning`

```

inline evaluate_move(m,in_alpha,in_depth,out_score,out_alpha)
{
  if
  :: out_score = NO_EVAL
  :: (in_alpha == NO_EVAL || in_depth == DEPTH_REDUCED) ->
    select(out_score : 1 .. SCORE_MAX)
  :: (in_alpha != NO_EVAL && in_depth == DEPTH_FULL) ->
    out_score = true_score[m]
  fi;
  if
  :: out_score > in_alpha -> out_alpha = out_score
  :: out_score <= in_alpha -> out_alpha = in_alpha
  fi
}

```

`NO_EVAL` stands for `INT_MAX`, defined as 9, larger than every genuine score in the abstract domain `1..SCORE_MAX`, which reproduces the one property of `INT_MAX` that matters here: it wins every comparison the master makes. `true_score[]` is verification state with no counterpart in the implementation. It fixes a genuine evaluation for each move at the start of a round, chosen non-deterministically, so that a property can compare the move the master played against the move it should have played; the implementation has no such array, since the true value of a move exists only implicitly, as whatever a correct full search would return.

Two omissions in Table 6 are not free, and both are central to Section 2.3.

One refinement was forced by Promela semantics. `select` cannot bind directly to an array element, so `true_score[i]` cannot appear as a `select` target; the model instead selects into a scratch variable and assigns it, the same workaround used nowhere in the scheduler but needed here because the scheduler has no array indexed by a loop variable at the point of a `select`. One correctness note also worth stating plainly: `bcast_terminate` sends only the colour broadcast with a negative value and not the board broadcast, because a worker breaks out of its loop before reaching the board receive, so the collective sequences on both sides still match.

Limits on interpretation

As with Program 1, this model was written by hand, so a counterexample is a candidate defect until checked against the source, and a property that holds establishes nothing outside the worker, move and round counts it was checked at. One further limit is specific to this model. Both the `MPI_Iprobe` busy-wait in `execute_master` and the `while (running)` loop in `run_master` are spin loops in the implementation; the model replaces both with blocking channel receives. That changes what Spin reports for the same underlying failure: a spinning `MPI_Iprobe` that never receives anything is an infinite execution in the implementation, which Spin would only find as a non-progress cycle and only with -1, whereas a blocking receive on a channel nothing will ever write to is an invalid end state, caught by the default safety check. Both mean the master never returns, so the abstraction turns an implementation livelock into a model deadlock, cheaper to

Table 6: What the model leaves out, and what each omission costs.

Omitted	Cost
Board contents, geometry, <code>is_stable</code> , board weights	None. No property depends on where a piece sits, only on the score a search returns for it.
Recursion inside <code>minimax</code> below the top call	None. It is thread-local and cannot affect any other process’s interleaving.
The <code>num_moves</code> -to-depth mapping (11, 13, 12 or 14)	None. Nothing in the protocol depends on the specific depth, only on whether a search was cut short, so all four collapse to <code>DEPTH_FULL</code> and <code>DEPTH_REDUCED</code> .
The <code>time()</code> cutoff clock	Small. A free nondeterministic choice can fire the cutoff at any point in a round, which over-approximates a real clock rather than modelling one; a counter would multiply the state space for no benefit to the properties checked.
The <code>MPI_Iprobe</code> busy-wait, replaced by a blocking receive	Not free. This changes what an unanswered result looks like to Spin, discussed in Section 2.1.

detect and with a shorter counterexample, and Section 2.3 reports it on that understanding rather than as a literal match to the C control flow.

2.2 Correctness Properties

Three variables have no counterpart in `my_player.c` and exist only so a property can be stated, the same role `executing[]` and `in_ready[]` play in the scheduler model. `round_done` is false while a round is in progress and true once the collection loop has finished and `best_move` is final. `best_true` is the largest `true_score` among the round’s moves. `workers_done` counts how many workers have left their loop and can reach `MPI_Finalize`.

Five checks were run: four properties as named ltl blocks, plus a deadlock check that is not an LTL property. Table 7 gives each formula as written in `model.pml`.

Table 7: The four LTL properties as written in `model.pml`.

Property	Class	Formula
<code>no_eval_chosen</code>	safety	<code>[] !(round_done && best_score == NO_EVAL)</code>
<code>best_is_maximal</code>	safety	<code>[] ((round_done && num_moves > 0) -> true_score[best_move] == best_true)</code>
<code>no_lost_results</code>	safety	<code>[] (round_done -> len(to_master) == 0)</code>
<code>workers_finish</code>	liveness	<code><> (workers_done == NUM_WORKERS)</code>

`no_eval_chosen` targets the leaf case of `minimax`’s minimising branch directly: there is no guard on `number_of_moves == 0`, so a node with no legal moves, an ordinary pass rather than an error, returns `minEval` still holding its initial `INT_MAX`. That value reaches `execute_master`’s comparison if `((process_score > max_score) || (max_move == -1))`, and since `INT_MAX` exceeds every real evaluation, the move that was never searched always wins. This is a correctness property of the value the engine plays, not merely an internal invariant.

`best_is_maximal` asks whether the move `execute_master` selects is really one of the best available; `num_moves > 0` excludes the pass case, where `best_move` is 0 and `true_score[0]` would

be meaningless. Two independent things can break it. The first is the `NO_EVAL` leak above. The second is that the master compares scores computed under different conditions: the search depth drops from 11 to 4 partway through a round once the turn clock passes `cut_off`, and the shared `alpha` means a worker that receives a high `alpha` prunes earlier than one that receives `INT_MIN`. Because the pruning test `if (beta <= *alpha) break;` sits at the *bottom* of the loop rather than the top, a poisoned or depth-reduced call still evaluates its first child in full and returns that child's ordinary-looking, in-range value instead of `INT_MAX`, which is why this second cause is silent in a way the first is not. The fault injection results isolating the two are in Section 2.3.

`no_lost_results` is the property the whole work-sharing protocol rests on, and the one expected to hold. `to_master` stands for the `MPI_Iprobe` plus `MPI_Recv(MPI_ANY_SOURCE)` pair, and a single shared channel is the right abstraction because `MPI_ANY_SOURCE` does not distinguish senders [6]. The `Iprobe` loop has no escape: the time cutoff lowers `depth` but never breaks it, so if the send and receive counts were ever unbalanced the master would spin forever. This property confirms the balance is exact.

`workers_finish` asks whether every worker eventually leaves its loop and can reach `MPI_Finalize`. A worker exits only on a negative broadcast colour, and only the `GAME_TERMINATION` and `UNKNOWN` branches of `run_master` issue that broadcast. `receive_message` in `comms.c` can also return `RECV_FAILED` or `CLIENT_DISCONNECTED`, and `run_master`'s if-chain has no branch for either; since `comms.h` is an interface the project specification fixed and forbade changing, handling all of its values is squarely the player's responsibility. Unlike the scheduler's liveness failures, this one fails both with and without `-f`: the cause is a missing branch, not a scheduling artefact, so the counterexample is genuine at either setting.

Deadlock is checked the same way as in Program 1, with a claim-free verifier compiled with `-DNOCLAIM`, since `pan` disables the invalid end state check while a never claim is active [2]. It asks whether any reachable state has a process blocked with no executable statement while the system has stopped making progress. This is worth stating alongside `workers_finish` rather than instead of it, because the two catch the same underlying defect through different formulations: one says the workers never finish, the other says the system halted somewhere it should not have.

Two things were considered and not verified. A property comparing the round number a worker receives against `round_no` was drafted to check that the two broadcasts, `bcast_colour` and `bcast_board`, stay matched; they are deliberately kept as two channels rather than merged into one so that such a mismatch would be observable, and this remains a natural extension rather than a checked result. Alpha monotonicity within a round, that `master_alpha` never decreases, was also considered and dropped: it is true by construction in both the model and the implementation, since `alpha` is only ever raised by a comparison, so a property asserting it would hold trivially and establish nothing.

2.3 Verification Results and Analysis

Each run selects one claim with `./pan -a -N name`, and the deadlock check uses a second binary built with `-DNOCLAIM` from the same generated `pan.c`, exactly as in Program 1 [2]. `NUM_WORKERS`, `MAX_MOVES` and `MAX_ROUNDS` are supplied to `spin` on the command line, and both binaries are rebuilt after every `spin -a`, since a stale binary silently reports results for the wrong configuration. All runs were on Spin 6.5.2 on the same 7 GB machine used for Program 1. iSpin was not used.

W, M and R abbreviate `NUM_WORKERS`, `MAX_MOVES` and `MAX_ROUNDS`. W3, M4, R1 matches the deployed system; W2, M3, R1 is the reduced configuration, small enough to search exhaustively, used wherever a property must hold rather than merely fail fast.

Table 8: Verification results, Spin 6.5.2. Depths and state counts as reported by `pan`.

Property	Config	Result	Depth	States
<code>no_eval_chosen</code>	W3, M4, R1	violated	205	5 336
<code>best_is_maximal</code>	W3, M4, R1	violated	255	106 138
<code>workers_finish</code>	W3, M4, R1	violated	31	203
<code>deadlock</code>	W3, M4, R1, -DNOCLAIM	violated	20	205
<code>no_eval_chosen</code>	W2, M3, R1	violated	172	1 046
<code>best_is_maximal</code>	W2, M3, R1	violated	222	17 286
<code>workers_finish</code>	W2, M3, R1	violated	28	77
<code>deadlock</code>	W2, M3, R1, -DNOCLAIM	violated	17	79
<code>no_lost_results</code>	W2, M3, R1	holds	n/a	7 676 089
<code>no_lost_results</code>	W3, M4, R1, -DBITSTATE	0 errors, approximate	436	655 692 780

The NO_EVAL leak

`no_eval_chosen` fails at depth 205 with a single worker hitting a node with no legal moves, small for the same reason `no_self_wait` was small in Program 1: it is a plain logic error, not a race. The defect is not confined to the first round either; at `MAX_ROUNDS 2` the same property fails again at depth 341 after 5683 states, past the point where the first round has already closed. The match logs in `othello/evidence/` confirm this happens in practice, not only in the model: 22 of the 60 rounds in match 20250513_115728 finalised on a score of 2147483647, and workers received an `alpha` of that value 76 times over the same match.

Two independent causes of a suboptimal move

`best_is_maximal` fails at depth 255, saying the engine played a worse move than one it had already evaluated, the same shape as round 47 versus round 24 in the match log, where move 47 scored 1328 and move 24 scored `INT_MAX` and was played anyway. Section 2.2 gives two mechanisms that can cause this on their own; the fault injection runs in Table 9 isolate them.

Table 9: Fault isolation on `best_is_maximal`, W2, M3, R1.

Flags	Result	Depth	States
none	violated	222	17 286
-DNO_TIME_CUTOFF	violated	218	10 026
-DNO_LEAK	violated	280	47 056
-DNO_TIME_CUTOFF -DNO_LEAK	holds	n/a	626 037

Removing either cause alone leaves the property violated; only removing both together makes it hold. That confirms the two mechanisms are independent: the `NO_EVAL` leak is sufficient to produce a suboptimal choice on its own, and so is comparing scores computed at different depths or under different `alpha` windows, so guarding `minimax` against the leak alone would not have fixed the property.

The missing message-type branch

`workers_finish` fails at depth 31, and the deadlock check finds an invalid end state at depth 20 in the same configuration; both trace to the same missing branch in `run_master` for `RECV_FAILED` and `CLIENT_DISCONNECTED`. The smallest witness needs no move round at all: at `MAX_ROUNDS 0` the deadlock still appears at depth 20 after 205 states and `workers_finish` still fails at depth 31 after 203 states, identical to the `MAX_ROUNDS 1` numbers, which shows the defect sits entirely in

the referee message loop and has nothing to do with move evaluation. Unlike Program 1’s liveness properties, `workers_finish` fails both with and without `-f`: the cause is a genuinely missing branch, not a scheduling artefact that fairness would explain away, so no fairness argument is needed here the way one was for `all_terminate` and `ready_dispatched`.

The property that probably holds

`no_lost_results` was checked exhaustively at W2, M3, R1 with 0 errors over 7 676 089 states in 11.4 seconds, with zero unreachable statements in all four proctypes. At the deployed configuration, W3, M4, R1, the same exhaustive search was killed by the kernel’s out-of-memory killer: the state vector is 244 bytes, so 7 GB holds roughly 2.6×10^7 states, and the search passed that without terminating. The fallback is bitstate storage, a compile-time option that replaces exact storage with a hash-indexed bit array so that distinct states can collide and be treated as already visited [?]. That run found 0 errors over 655 692 780 states in around 1 370 seconds, but its hash factor of 6.55 is well below the 100 that indicates near-exhaustive coverage, so an appreciable fraction of the state space may have gone unexplored. The correct claim is that no counterexample was found in an approximate search of roughly 6.6×10^8 states, not that the property holds at three workers; the exhaustive two-worker result is the stronger evidence, and both are reported here rather than only the more flattering one.

The reduction to two workers is defensible for this specific property. A lost result requires the master to close a round while a message it should have consumed is still queued, and the shortest such witness needs only one worker whose result is accounted for and one whose result is not, which two workers already provide; a third worker would only admit longer instances of the same shape. That is an argument about the shortest witness, not a proof, and it establishes absence only within the verified bound.

One search-order detail is worth recording because it affects how large these numbers look without changing what they mean. `proctype Referee`’s four terminal message branches are listed before its `budget > 0` branch. Depth-first search explores a nondeterministic `do` in the order written, so with the terminal branches first, a termination counterexample is found in a few hundred states rather than after an entire move round has already been explored, which is where the tens-of-millions-of-states figures in earlier drafts of this model came from. This changes only the order the search visits the state space, not the space itself: `no_lost_results` at W2, M3, R1 reports exactly 7 676 089 states before and after the reordering, since that property’s search must be exhaustive either way and visits every state regardless of order.

Proposed fixes

Each of the three defects needs a different kind of fix.

The NO_EVAL leak is the easiest to close. The minimising branch of `minimax` needs a guard on `number_of_moves == 0` before the loop, because a node with no legal moves is an ordinary pass in Othello and not an error. What it should return there is a real evaluation of the current position, which the engine already has in `evaluate`, rather than the `INT_MAX` the accumulator was initialised to. The maximising branch has the same shape with `INT_MIN`, so both need the guard even though only the minimising one showed up here. A cheap second layer is to reject a score equal to `INT_MAX` in `execute_master` before it is compared against `max_score`, so that a leak from anywhere else in the search cannot reach the shared `alpha`.

The comparability defect is a design change rather than a missing test. The master lowers `depth` from 11 to 4 in the middle of a round, then compares results computed on either side of that change as though they were on the same scale. The smallest correct version is to apply the new depth only from the next round, so that every score within a round comes from a search of the same depth. If the turn clock genuinely cannot wait, the alternative is to tag each result with the depth it was computed at and keep the best move from the deepest completed depth,

discarding the shallower results instead of mixing them in. Table 9 is what makes this worth the effort: fixing the leak alone leaves `best_is_maximal` violated.

The missing message branch is the smallest fix of the three, and the safer form of it is not to add two more `else if` arms for `RECV_FAILED` and `CLIENT_DISCONNECTED`, but to give the if-chain a final `else` that sets `running = 0` and issues the termination broadcast. Naming the two values explicitly fixes today’s defect; a default arm also covers any value `comms.h` might gain later, which matters because the interface is fixed by the framework and the player cannot change it. Either way the workers stop blocking in `MPI_Bcast` and reach `MPI_Finalize`, which is what `workers_finish` asks for.

A Reproducing the verification results

Table 10: Environment for every run reported in this document.

Model checker	Spin Version 6.5.2, 6 December 2019
Compiler	gcc
Platform	Ubuntu Linux
Memory	7 GB RAM, 1 GB swap

Table 10 gives the environment. iSpin was not used for any result in either program, and all runs were performed from the command line.

Program 1: FCFS scheduler

Model version: commit 517e9be.

Listing 4: Build

```
cd Modelchecking/scheduler
spin -a model.pml
gcc -O2 -o pan pan.c
gcc -O2 -DNOCLAIM -o pan_nocclaim pan.c
```

Listing 5: Verification runs, default configuration (MAX_INSTR 3)

```
./pan -a -N pcb_mutex
./pan -a -N no_self_wait
./pan -a -N waiting_consistent
./pan -a -N all_terminate -m50000
./pan -a -N all_terminate -f -m50000
./pan -a -N ready_dispatched -m50000
./pan -a -N ready_dispatched -f -m50000 -w26
./pan_nocclaim
```

Listing 6: Reduced configuration (MAX_INSTR 2), for the property that holds

```
spin -a -DMAX_INSTR=2 model.pml
gcc -O2 -o pan pan.c
./pan -a -N no_dup_ready -m20000 -w26
```

Program 2: Othello

Model version: commit 70b73d2.

Every result in Table 8 and Table 9, except the three-worker `no_lost_results` run below, is produced by a single script:

Listing 7: Build and run everything

```
cd Modelchecking/othello
./verify.sh
```

This regenerates `pan.c` and rebuilds both binaries before each group of runs, writes every error trail to `results/`, the full `pan` output of every run to `results/logs/`, and a summary table to `results/summary.txt`. Passing `quick` as the first argument skips the two exhaustive runs (configurations D and E) for a faster, partial regeneration.

The three-worker exhaustive `no_lost_results` run is excluded from the script because it exceeds available memory; its bitstate fallback takes roughly 23 minutes and is a deliberate one-off rather than a routine result, so it is run manually:

Listing 8: Three-worker `no_lost_results`, bitstate fallback

```
spin -DNUM_WORKERS=3 -DMAX_MOVES=4 -DMAX_ROUNDS=1 -a model.pml
gcc -O2 -DBITSTATE -o pan_bit pan.c
./pan_bit -a -N no_lost_results -m100000 -w32
```

References

- [1] Gerard J. Holzmann. Promela reference: `atomic(3)`. <https://spinroot.com/spin/Man/atomic.html>, 2025. Accessed 2025.
- [2] Gerard J. Holzmann. Pan verification options. <https://spinroot.com/spin/Man/Pan.html>, 2025. Accessed 2025.
- [3] Cornelia P. Inggs. Promela. Lecture slides, CS796 Software Verification and Spin Model Checking, Stellenbosch University, 2025. Source: The Spin Model Checker, G. J. Holzmann.
- [4] Cornelia P. Inggs. Properties. Lecture slides, CS796 Software Verification and Spin Model Checking, Stellenbosch University, 2025. Source: The Spin Model Checker, G. J. Holzmann.
- [5] Cornelia P. Inggs. Spin. Lecture slides, CS796 Software Verification and Spin Model Checking, Stellenbosch University, 2025. Source: The Spin Model Checker, G. J. Holzmann.
- [6] Message Passing Interface Forum. MPI: A message-passing interface standard, version 4.1. <https://www.mpi-forum.org/docs/>, 2023.
- [7] OpenMP Architecture Review Board. OpenMP application programming interface, version 5.2. <https://www.openmp.org/specifications/>, 2021.
- [8] Abraham Silberschatz, Peter B. Galvin, and Greg Gagne. *Operating System Concepts*. Wiley, 10th edition, 2018.